

Kvalitetsplan

Grupp 13

2026-03-02

Version 0.1

Status

Granskad	Szilvia Varro-Gyapay	2026-02-20
Godkänd	Szilvia Varro-Gyapay	2026-xx-xx

Projektidentitet

Grupp E-post: tddd96_2026vt_ax_13@groups.liu.se

Beställare: Johan Munck, Börjes Koncernen
Tfn: +46733318552
E-post: johan@borjeskoncernen.se

Handledare: Szilvia Varro-Gyapay
Tfn: +4613284092
E-post: szilvia.varro-gyapay@liu.se

Kursansvarig: Lena Buffoni
Tfn: +4613284046
E-post: lena.buffoni@liu.se

Projektdeltagare

Namn	Ansvar	E-post
Karl Lennartsson	Teamledare	karle221@student.liu.se
Simon Andersson	Dokumentansvarig	siman020@student.liu.se
Elias Facq	Analysansvarig	eliwa076@student.liu.se
Jacob Ahling Hult	Utvecklingsledare	jacah067@student.liu.se
Jakob Renqvist	Arkitekt	jakre732@student.liu.se
Viktor Wallberg	Kvalitetssamordnare	vikwa448@student.liu.se
Felix Hallström	Testledare	felha291@student.liu.se

INNEHÅLL

1	Inledning	1
1.1	Beställare	1
1.2	Bakgrund	1
1.3	Syfte	1
1.4	Relaterade dokument	1
1.5	Kvalitetsmål	1
2	Mätprogram	2
2.1	Mätning för kvalitetsmål 1	2
2.2	Mätning för kvalitetsmål 2	3
2.3	Mätning för Kvalitetsmål 3	3
2.4	Mätning för kvalitetsmål 4	4
3	Organisation	4
3.1	Roller och Ansvarsområden	4
3.2	Teamarbete	5
3.3	Utbildning	5
3.4	Gruppkontrakt	5
4	Arbetsmetodik	5
4.1	Möten	5
4.2	Kommunikation inom gruppen	6
4.3	Extern kommunikation	6
4.4	Uppföljning och förbättring	6
4.5	Git	6
4.6	Riskhantering	7
4.7	Riskregister	7
4.8	Riskprocess	7
5	Dokumentkvalitet	7
5.1	Dokumentstandard	8
5.2	Dokumentgranskning	8
5.3	Dokumentöversikt	9
5.4	Dokumentändringar och kompletteringar	9
6	Mjukvarukvalitet	10
6.1	Teknisk plattform	10
6.2	Kodstandard	10
6.3	Kodgranskning	11
6.4	Release och deploy (Vercel)	12
6.5	Datakvalitet och databasförändringar (Supabase)	12
7	Referenser	13

DOKUMENTHISTORIK

Version	Datum	Utförda förändringar	Utförda av	Granskad
0.1	2026-02-18	Första utkast	Viktor Wall-berg	2026-02-18
0.2	2026-02-20	Uppdaterad utifrån granskning	Viktor Wall-berg	
1.0		Första version	Grupp 13	

1 INLEDNING

Den här delen ger en överblick av projektet och hur kvaliteten av systemet säkerställs.

1.1 Beställare

Beställaren i detta projekt är Johan Munck från Börjes Koncernen.

1.2 Bakgrund

Varje transportrutt i nuläget fungerar kort förklarad genom att en trafikledare planerar en rutt för en leverans, vilken en lastbilsförare sedan godkänner. På det sättet finns det inget sätt att med säkerhet veta i förhand huruvida den planerade rutten kommer vara lönsam eller inte, utan det kan enbart beräknas i efterhand. Det gör att rutterna som planeras av trafikledare grundar sig i erfarenhet från tidigare planerade rutter, men det är inte alltid lönsamt eftersom marginalen ligger på 5% och det tar väldigt lång tid att lära sig.

1.3 Syfte

Projektets syfte är att bygga ett system som trafikledare inom Börjes koncern kan använda för att effektivare avgöra om en transportrutt är lönsam eller inte utifrån historisk data.

1.4 Relaterade dokument

- Projektplan
- Kravspecifikation
- Roller och dokument (Föreläsning)
- Dokumentvägledning

1.5 Kvalitetsmål

1.5.1 Kvalitetsmål 1

- **Mål:** Eftersom trafikledarna kommer använda systemet i en hektisk miljö avser vi att systemet ska vara användarvänligt med ett intuitivt gränssnitt.
- **Användarvänligt:** Systemet ska vara enkelt att använda och gränssnittet ska vara enkelt att förstå och navigera i.
- **Mätbara krav:** Systemet ska uppnå $SUS \geq 70$ (System Usability Scale).

1.5.2 Kvalitetsmål 2

- **Mål:** Programmet ska fungera i drift för trafikledare även om det uppstår interna eller externa fel i systemet.

- **Mätbara krav**

- Prestanda: En lönsamhetsberäkning ska ta högst 3 sekunder att genomföra.
- Robusthet: Systemet ska hantera felaktig inmatning och externa fel (t.ex. API-fel, saknad data) utan krasch och med begripligt felmeddelande.

1.5.3 Kvalitetsmål 3

- **Mål:** Systemet ska vara skalbart utan omfattande ombyggnad.
- **Skalbarhet:** Systemet ska vara konstruerat för att man enkelt ska kunna vidareutveckla och skala systemet genom att lägga till eller uppdatera funktionalitet.
- **Mätbara krav:**
 - Arkitekturen ska uppfylla definierade arkitekturprinciper (lagerseparation, tydliga gränssnitt, moduläritet) enligt arkitekturgranskning.

1.5.4 Kvalitetsmål 4

- **Mål:** Systemets databaser ska vara skyddade från läckage och intrång.
- **Säkerhet:** Eftersom prislistor och likartad information är känslig måste systemets databaser vara säkra och skyddade.
- **Mätbara krav**
 - Behörighets- och åtkomstkontroller ska fungera enligt definierade testfall.
 - Säkerhetskrav enligt checklista ska vara uppfyllda.

2 MÄTPROGRAM

Mätprogrammet beskriver hur de mätbara kraven i avsnitt 1.4 verifieras: mätmetod, acceptanskriterier, när det mäts, ansvar och var resultat dokumenteras.

2.1 Mätning för kvalitetsmål 1

- Hur mäts det: SUS-enkät efter standardiserat testschema (definierat scenarionflöde).
- Acceptanskriterium: $SUS \geq 70$.
- Ansvar: Testledare + kvalitetssamordnare.
- Artefakt: SUS-enkäter + sammanställning i testrapport/statusrapport.

2.2 Mätning för kvalitetsmål 2

A) Prestanda

- Hur mäts det: Prestandatest med definierat dataset och upprepade körningar (samma testfall varje gång). Mät svarstid för beräkningen (backend eller end-to-end, enligt er testdesign).
- Acceptanskriterium: Den 95:e percentilen av svarstiden ska uppfylla $P_{95} \leq 3\text{sek}$.
- När: Vid större ändringar i beräkningslogik/integration samt inför större leveranser.
- Ansvar: Utvecklingsledare + testledare.
- Artefakt: Prestandaloggar + sammanställning.

B) Robusthet

- Krav: Systemet ska inte krascha vid felaktig inmatning eller externa fel och ska ge begripligt felmeddelande.
- Hur mäts det: Definierad uppsättning negativa testfall (ogiltig inmatning, saknad data, externa API-fel). Varje testfall ska ge kontrollerat fel och felmeddelande.
- Acceptanskriterium: 100% av definierade negativa testfall uppfyller kravet (ingen krasch, kontrollerat fel, begripligt fel).
- När: Löpande (minst vid omfattande ändringar) samt inför varje större leverans.
- Ansvar: Testledare.
- Artefakt: Testprotokoll + incident-/avvikelselista.

2.3 Mätning för Kvalitetsmål 3

- Krav: Arkitekturen ska uppfylla definierade arkitekturprinciper.
- Hur mäts det: Arkitektur/designreview med checklista som täcker lagerseparation, moduläritet och gränssnitt.
- Acceptanskriterium: Samtliga kärnprinciper är uppfyllda, eller (om avvikelse finns) dokumenterad avvikelse med åtgärdsplan och deadline.
- När: Vid större arkitekturändringar och inför relevanta leveranser.
- Ansvar: Arkitekt + utvecklingsledare.
- Artefakt: Review-protokoll + uppdaterat arkitekturdokument.

2.4 Mätning för kvalitetsmål 4

- Krav: Databaser och åtkomst ska vara skyddade från läckage och intrång.
- Hur mäts det: behörighets-/åtkomsttester (positiva och negativa), säkerhetschecklista (t.ex. autentisering, rättigheter, säker dataöverföring).
- Acceptanskriterium: 100% pass på behörighetstester samt samtliga säkerhetskrav i checklistan uppfylla.
- När: Vid ändringar i autentisering/behörighet/integration samt inför leverans.
- Ansvar: Utvecklingsledare + testledare.
- Artefakt: Testprotokoll + checklista.

3 ORGANISATION

I denna del beskrivs organisationen av teamet och vad som förväntas av varje individ.

3.1 Roller och Ansvarsområden

Teamet som kommer arbeta med detta projekt består av sju personer som alla har varin roll med innefattande ansvarsområde. Rollerna inkluderar:

- Teamledaren har ansvaret att se till att projektet tar sig framåt, informera resterande gruppen med information från examinator och handledare. Teamledaren representerar teamet utåt och har också det sista ordet i alla frågor angående projektet
- Analysansvarig innebär att vara primära personen för kundkontakt. Den ansvarige ska ta reda på kundens krav, even- tuellt förhandla och förmedla information till gruppen.
- Utvecklingsledaren ansvarar för utvecklingsarbetet och fattar beslut om utvecklingsmiljöer. Utvecklingsledaren ansvarar för en detaljerad design.
- Arkitekten ser till att en stabil arkitektur tas fram genom att identifiera komponenter och gränssnitt, samt dokumentera denna.
- Testledaren beslutar om systemets status och sköter den dynamiska verifieringen och valideringen av systemet genom exekvering. Testledaren skriver även testplanen och testrapporterar.
- Den som är Dokumentansvarig ser till att dokumentmallar och ändringsrutiner följs och ser även till att leveranser till deadlines följs.
- Kvalitetssamordnaren ser till att ett kvalitativt arbete utförs av alla. Även innebär rollen att ansvara över utbildning och inspiration för gruppen, samt planera och budgetera ihop med övriga gruppen.

(Grupp 13, 2026)

Förtydligande om beslutsmandat:

- Teamledare har sista ordet om beslut behöver fattas snabbt eller vid låsning.
- Arkitekt har (näst) sista ordet i tekniska arkitektur- och strukturfrågor.
- Utvecklingsledare fattar beslut om utvecklingsmiljö och driver detaljerad design.
- Testledare fattar beslut om teststatus och acceptans inför leveranser.
- Dokumentansvarig ansvarar för att interna deadlines och granskningsprocesser hålls.
- Kvalitetssamordnare följer upp mätningar och säkerställer att åtgärder initieras vid avvikelser.

(Linköpings universitet, 2026)

3.2 Teamarbete

Formella roller används för att tydliggöra ansvarsfördelningen. Samtidigt förväntas samtliga teammedlemmar i möjligaste mån bistå varandra och bidra även utanför det egna ansvarsområdet.

3.3 Utbildning

Varje individ ansvarar i första hand för att sätta sig in i sitt ansvarsområde samt vid behov lära sig nya system, verktyg och programmeringsspråk. Om en teammedlem stöter på problem eller svårigheter kan personen vända sig till övriga teammedlemmar eller till kvalitetsansvarig, som kan hjälpa till att hitta relevant kompetens inom teamet.

3.4 Gruppkontrakt

Vi har i gruppen upprättat ett gruppkontrakt för att tydliggöra förväntningar och ansvar, säkerställa att samarbetet fungerar väl och ha en gemensam grund att utgå från om problem uppstår.

4 ARBETSMETODIK

Projektet bedrivs iterativt med tydliga delmål. Arbetet planeras löpande, prioriteringar uppdateras vid behov och avvikelser följs upp så att leveranser håller tid och kvalitet.

4.1 Möten

Vi genomför regelbundna avstämningar för planering, uppföljning och problemlösning. Varje möte har agenda, tydliga beslut/åtgärder och en ansvarig som dokumenterar korta anteckningar samt uppföljningspunkter. För att följa upp tiden för mötet avser gruppen en person att notera klockan vid start och slut av mötet. Mötesanteckningar sparas i gruppens gemensamma Google Drive. Teamledaren ansvarar i första hand för att föra anteckningar, inklusive beslut och åtgärds punkter (ansvarig och deadline). Vid behov kan teamledaren ge ansvaret att föra anteckningar till annan deltagare.

4.2 Kommunikation inom gruppen

Intern kommunikation sker via gemensamma kanaler där Messenger används för snabb kommunikation och Discord för bredare information och samlad projektkommunikation. Viktiga beslut sammanfattas skriftligt så att alla har samma bild och för att undvika missförstånd.

4.3 Extern kommunikation

Extern kommunikation med kund sker i första hand via analysansvarig till kontaktperson för att säkerställa tydlighet. Frågor, svar, beslut och ändringar dokumenteras kort och delas med berörda i teamet. Alla i teamet har däremot möjlighet att ta kontakt med kund vid mer personliga frågor. Teams används främst för inlämning och formell kommunikation med handledare.

4.4 Uppföljning och förbättring

Syftet med uppföljning är att tidigt upptäcka avvikelser och säkerställa kontinuerliga förbättringar i både arbetssätt och produkt.

4.4.1 Löpande uppföljning

- **Veckovis uppföljning:** Vid veckomötet följs plan vs. utfall upp, risker går igenom, samt status för mätprogrammet (se avsnitt 2).
- **Avvikelsehantering:** Om en mätning inte uppnår acceptanskriteriet skapas en åtgärd i backloggen. Åtgärden ska ha ansvarig, deadline och en plan för ommätning.
- **Spårbarhet:** Resultat och åtgärder dokumenteras kort i statusrapport/testrapport och länkas till relevant arbetsuppgift.

4.4.2 Förbättringsprocess

- **Frekvens:** Varannan vecka (eller efter större leverans).
- **Genomförande:** Gruppen reflekterar kring vad som fungerat bra, vad som fungerat mindre bra och väljer eventuella förbättringsåtgärder.

4.4.3 Hur vi mäter progression i projektet

- **Leveransprogress:** Uppföljning av färdiga arbetsuppgifter mot plan (kursens och eventuellt beställares).
- **Kvalitetsprogress:** Uppföljning av mätprogrammet.

4.5 Git

Vi använder GitHub för versionshantering med en tydlig branching-strategi och pull requests för ändringar.

Vad Git/GitHub används till: ·

- versionshantering och spårbarhet (koppling mellan ändring och uppgift/ärende),
- möjliggöra kodgranskning via pull requests,
- hålla main/master i körbart skick,
- stödja releaser och deploy (kopplat till Vercel).

Grundregler:

- arbete sker i separata brancher,
- commitmeddelanden ska vara beskrivande och kopplas till uppgift/ärende,
- merge sker via pull request,
- main/master hålls alltid i körbart skick.

Koppling till kvalitetsarbete

- Pull requests ska innehålla kort beskrivning, koppling till krav/issue och hur ändringen testats.
- Vid förändringar som påverkar kvalitetskrav (t.ex. prestanda, robusthet, säkerhet) ska relevant mätning/test uppdateras och dokumenteras.

4.6 Riskhantering

Syftet är att proaktivt hantera risker som kan påverka tid, kvalitet eller leverans.

4.7 Riskregister

Riskerna utgår från projektplanens riskanalys (R1–R7). Varje risk har sannolikhet, påverkan, begränsning (mitigation), beredskap (contingency), riskägare och trigger.

4.8 Riskprocess

1. Identifiera risker (möten/Discord/Messenger/issue).
2. Bedöm sannolikhet och påverkan.
3. Bestäm begränsning och beredskap samt ansvarig.
4. Följ upp veckovis i samband med veckomöte/statusrapport.
5. Eskalera vid risk som hotar leverans/tollgate.

5 DOKUMENTKVALITET

Dokument ska vara konsistenta, lättlästa och spårbara. Ändringar ska kunna följas och dokument ska hålla en kvalitet som gör dem användbara både under projektet och vid överlämning.

5.1 Dokumentstandard

Dokument följer en gemensam mall med rubriker, versionshistorik, datum och ansvarig. Språk ska vara tydligt och formellt, begrepp konsekventa och eventuella förkortningar förklaras. Dokument uppdateras iterativt under projektets gång.

Utöver den gemensamma mallen utgår varje dokument från rekommenderad standard/struktur:

- **Projektplan:** Rubriker enligt kursupplägg från TDDC93 (PUM-struktur).
- **Kvalitetsplan:** IEEE Std 730.
- **Kravspecifikation:** IEEE Std 830.
- **Arkitekturdokument:** OpenUP *Architectural Notebook*.
- **Testplan:** ISO/IEC/IEEE 29119-3 (kapitel 6–7) samt IEEE Std 829.

(Linköping universitet, 2026)

5.2 Dokumentgranskning

Alla dokument granskas av minst en annan teammedlem innan de fastställs. Granskning fokuserar på korrekthet, tydlighet, konsekvens och att innehållet matchar krav och beslut.

5.2.1 Granskningsprocess

1. **Initiering:** Dokumentförfattaren startar granskning genom att kontakta gruppen/granskare att dokumentet är redo för granskning, vem som är granskare för respektive dokument kommer diskuteras fram i gruppen vid granskningstillfällen.
2. **Granskning:** Granskare använder en checklista som kommer tas fram i gruppen, exempelvis:
 - spårbarhet mot krav/beslut,
 - tydlighet och struktur,
 - konsekvens i begrepp och formuleringar,
 - uppdaterad versionshistorik och ändringsbeskrivning.
3. **Åtgärd:** Författaren inför ändringar.
4. **Fastställning:** Dokumentansvarig verifierar att granskning är genomförd och att dokumentet är redo för inlämning.

5.2.2 Interna deadlines

- Dokumentutkast ska vara klart minst **48 timmar** före inlämning.
- Granskning ska vara klar minst **24 timmar** före inlämning.

5.3 Dokumentöversikt

Följande dokument är centrala i projektet och uppdateras iterativt under kursen:

- **Kvalitetsplan:** Beskriver hur kvalitet säkerställs (processer, mätning, uppföljning och förbättring).
- **Kravspecifikation:** Samlar och prioriterar krav (funktionella och icke-funktionella) och är grund för acceptans.
- **Projektplan:** Beskriver organisation, arbetssätt, leveranser/milstolpar, risker och rapportering.
- **Arkitekturdokument:** Beskriver arkitektur, komponenter, gränssnitt, teknikval och motiveringar.
- **Testplan:** Beskriver teststrategi, testnivåer, testfall/typer, ansvar, miljö och teststatus.

Dokumenterna lagras i Overleaf innan inlämning.

5.4 Dokumentändringar och kompletteringar

När en medlem ändrar, lägger till eller skriver nytt i något av dokumenten ska följande process användas för att behålla spårbarhet och konsekvens.

5.4.1 När ska en ändring göras?

- När ny information finns (t.ex. nya krav, nya risker, ny arkitektur, uppdaterad teststrategi).
- När handledare/beställare ger feedback.
- När mätningar/uppföljning visar att planen inte stämmer med verkligheten.

5.4.2 Ändringsprocess (steg för steg)

1. Initiera ändring

- Skapa en uppgift/issue för ändringen genom kontakt med gruppen/handledare/beställare vid behov (vad, varför, påverkan, deadline).
- Om ändringen påverkar flera dokument: ta kontakt med dokumentansvarig för konsistens vid uppdatering.

2. Genomför ändring

- Gör ändringen i Overleaf.
- Håll texten konsekvent med övriga dokument, meddela dokumentansvarig om något dokument måste uppdateras.

3. Uppdatera dokumentmetadata

- Uppdatera version, datum och författare i dokumenthistoriken.

4. Granskning

- Författaren initierar granskning och utser minst en granskare.

5. Stängning och kommunikation

- När granskning är åtgärdad och dokumentet är fastställt:
 - meddela i gruppens kanal att en ny version finns.

5.4.3 Konsistens mellan dokument

För att undvika motsägelser gäller:

- Ändras ett **krav** i Kravspec: kontrollera och uppdatera vid behov Kvalitetsplan (kvalitetsmål/mätprogram), Arkitekturdokument (om arkitekturval påverkas) och Testplan (verifiering).
- Ändras ett **kvalitetskrav/mätning** i Kvalitetsplan: säkerställ att motsvarande krav finns i Kravspec och att Testplan beskriver hur det testas.
- Ändras **arkitektur**: uppdatera Arkitekturdokument och kontrollera påverkan på Testplan och relevanta krav.

6 MJUKVARUKVALITET

Mjukvaran ska vara korrekt, stabil och underhållbar. Funktionalitet ska kunna verifieras genom testning och koden ska vara begriplig för fler än en enskild utvecklare i gruppen. Detta avsnitt beskriver teknisk plattform, kodstandard och processer för att säkerställa kvalitet i implementation, integration och leverans.

6.1 Teknisk plattform

Systemet utvecklas som en fullstack-lösning från start och kan vid behov kompletteras med en separat backend för ML-funktionalitet senare (t.ex. som en fristående server/tjänst).

- **Frontend:** Next.js (React), TypeScript, HTML och Tailwind CSS.
- **Databas/Backend-tjänster:** Supabase.
- **Deploy:** Vercel.
- **Versionshantering:** GitHub.

Kvalitetsimplikationer (vad vi säkrar):

- **Miljö och konfiguration:** Konfiguration och hemligheter hanteras via miljövariabler. Nycklar/hemligheter ska aldrig lagras i repository.
- **Spårbar leverans:** Varje deploy ska vara spårbar till en commit och helst till en pull request.
- **Databasändringar:** Ändringar i databas/schema ska göras kontrollerat och dokumenteras (t.ex. via migrations eller motsvarande rutin), så att förändringar är reproducerbara och kan granskas.
- **Separationsprincip:** Om en ML-backend tillkommer ska gränssnitt och dataflöden definieras tydligt så att kärnsystemet fortsatt är testbart och robust även vid extern tjänst.

6.2 Kodstandard

Kod ska vara konsekvent, lätt att läsa och underhålla samt bidra till att minska risken för fel vid förändringar.

6.2.1 Allmänna riktlinjer

- **Läsbarhet:** Använd tydliga namn, små funktioner/komponenter och undvik duplicerad logik.
- **Struktur:** Följ en gemensam projektstruktur (mappar och modulindelning) och separera UI, affärslogik och dataåtkomst.
- **Felhantering:** Fel ska hanteras kontrollerat med tydliga felmeddelanden till användare och loggningsstöd för utveckling/test.
- **Typer:** TypeScript-typer ska användas konsekvent för att minska runtime-fel och göra gränssnitt tydliga.
- **Dokumentation:** Icke-trivial logik och viktiga antaganden ska dokumenteras kort i kod eller i teknisk dokumentation.

6.2.2 Testbarhet

- Kod ska skrivas så att viktiga delar kan testas isolerat (t.ex. beräkningslogik, datakonvertering, validering).
- Kritiska flöden ska täckas av testfall definierade i testplanen (t.ex. positiva/negativa scenarion, fel i extern data).

6.2.3 Prestanda och robusthet i implementation

- Beräkningslogik ska implementeras med hänsyn till prestandakravet för lönsamhetsberäkning (mätning beskrivs i mätprogrammet).
- Externa beroenden (t.ex. datakällor) ska hanteras med timeouts och fallbacks där det är relevant, för att undvika att systemet blir obrukbart.

6.3 Kodgranskning

All ny omfattande kod som till större del har en påverkan går via pull request och granskas av minst en annan teammedlem innan merge. Syftet är att upptäcka fel tidigt och höja kvaliteten. För små ändringar som författaren inte anser har stor påverkan kan detta bortses.

6.3.1 Process

1. **Initiera:** Utvecklaren skapar en pull request och tilldelar minst en granskare.
2. **Beskrivning:** Pull request ska innehålla:
 - kort beskrivning av ändringen,
 - koppling till issue/krav (om relevant),
 - hur ändringen testats (t.ex. manuellt scenario, körda tester).
3. **Granska:** Granskaren kontrollerar:
 - korrekthet mot acceptanskriterier,
 - läsbarhet och struktur,

- felhantering och säkerhetsaspekter (vid behov),
- påverkan på prestanda och robusthet (vid behov),
- att relevanta tester finns/uppdaterats.

4. **Åtgärda:** Författaren åtgärdar synpunkter och uppdaterar pull request.

5. **Merge:** Merge sker först när granskning är klar och ändringen bedöms uppfylla kraven.

6.3.2 Regler

- **Main/master ska hållas körbar:** Ändringar som riskerar att bryta bygg eller grundfunktionalitet ska inte mergeas.
- **Kvalitetskrav:** Om en ändring påverkar kvalitetskrav (t.ex. prestanda, robusthet, säkerhet) ska relevant mätning-/test uppdateras.
- **Minimikrav för merge:** Minst en godkännande-granskning krävs innan merge.

6.4 Release och deploy (Vercel)

För att säkerställa kvalitet vid leverans ska deploy vara spårbar och reproducerbar.

- Deploy ska kunna kopplas till en specifik commit (och helst till en pull request).
- Miljövariabler och secrets ska hanteras via deploy-plattformen och får inte exponeras i repository eller loggar.
- Vid eventuella incidenter i deploy ska förändringen kunna isoleras genom att identifiera ansvarig commit/pull request och åtgärda via ny pull request.

6.5 Datakvalitet och databasförändringar (Supabase)

Databasen är en kritisk del av systemets funktion och kvalitet.

- Vid förändringar i dataformat eller tabeller ska påverkan analyseras för beräkningslogik och testfall.
- Åtkomst till databasen ska styras av behörighet och principen om minsta privilegium, och nycklar ska hanteras som hemligheter.

7 REFERENSER

- Linköpings universitet (2026) *Föreläsning 4: Roller och dokument*. Kurs TDDD96, VT 2026.
Tillgänglig på: https://liuonline.sharepoint.com/sites/Lisam_TDDD96_2026VT_AX/CourseDocuments/Forms/AllItems.aspx?viewid=48d69b64%2Dcfe7%2D4e92%2Db0a0%2D7407a448a46d&id=%2Fsites%2FLisam%5FTDDD96%5F2026VT%5FAX%2FCourseDocuments%2FF%C3%B6rel%C3%A4sningar%2F4%5Frollerdokument2026%2Epdf&parent=%2Fsites%2FLisam%5FTDDD96%5F2026VT%5FAX%2FCourseDocuments%2FF%C3%B6rel%C3%A4sningar
- Grupp 13 (2026), *TDDD96 Projektplan* (Version 0.1), Intern projektdokumentation, Linköpings universitet
- Linköpings universitet (2026) *DokumentVagledning*. Kurs TDDD96, VT 2026.
Tillgänglig på: https://liuonline.sharepoint.com/sites/Lisam_TDDD96_2026VT_AX/CourseDocuments/Forms/AllItems.aspx?id=%2Fsites%2FLisam%5FTDDD96%5F2026VT%5FAX%2FCourseDocuments%2FInl%C3%A4mningar